

SUMMER RESEARCH INTERNSHIP PROGRAMME 2026

FINAL RESEARCH REPORT — SUBMISSION COVER PAGE

A Hybrid Genetic Algorithm and Lion Optimization based Hybrid Task Offloading approach to Enhance Performance in MEC

PROGRAMME DETAILS

Domain / Track:	Cloud Computing / Mobile Edge Computing	Duration:	15/05/2026 – 14/08/2026
Department:	B.Tech CSE	Batch / Year:	3rd Year, 2026

SUBMITTED BY

Student 1	Soumyadeep Panja
Student 2	Mohit Kumar Ray
Student 3 (if applicable)	N/A

UNDER THE MENTORSHIP OF

Mentor Name	Gopa Mandal
Affiliation	Jalpaiguri Government Engineering College

DECLARATION

We hereby declare that this report is a record of genuine work carried out by us under the guidance of the above-mentioned mentor, as part of the IEEE SMC Student Branch Chapter, KGEC — Research Internship Programme 2026, and has not been submitted elsewhere for any other purpose.

Mentor's Signature

Programme Coordinator

IEEE SMC Student Branch Chapter, KGEC | Research Internship Programme 2026 | Date of Submission: 13 / 08 / 2026

Final Report of Internship Programme

Table of Contents

Acknowledgements	iii
Abstract	iv
List of Figures	v
List of Tables	vi
1 Introduction	1
2 Literature Review	2
2.1 Comparative Study of Twenty Relevant Studies	3
2.2 Inference Drawn from Literature Review	3
3 Problem Formulation and Proposed Solution	4
3.1 Overview	4
3.2 System Model	4
3.3 Task Properties and VM/Server Properties	5
3.4 Optimization Algorithms	5
3.5 Hybrid GA–Lion Approach	6
4 Results and Discussion	7
4.1 Experimental Setup	7
4.2 Performance Analysis	7
4.3 Statistical Analysis	8
4.4 Overall Discussion	9
5 Conclusion and Future Work	10
References	11
AI Usage Declaration	11

Final Report of Internship Programme

Acknowledgements

I express my sincere gratitude to Prof. Gopa Mandal, Jalpaiguri Government Engineering College, for her guidance, feedback, and support during the research internship. I also acknowledge the IEEE Systems, Man, and Cybernetics Student Branch Chapter at Kalyani Government Engineering College for providing the internship framework and research environment.

Abstract

Mobile Edge Computing (MEC) enables computational tasks to be processed closer to end users and data sources, but efficient task offloading remains challenging because of varying workload demands and limited edge resources. This work proposes a **Hybrid Genetic Algorithm and Lion Optimization based Hybrid Task Offloading approach** to enhance performance in MEC environments. The proposed approach combines the evolutionary search capability of Genetic Algorithm (GA) with the population-based search and refinement capability of Lion Optimization (Lion) for efficient task-offloading decisions. The proposed method is evaluated against five other approaches: Baseline, Particle Swarm Optimization (PSO), Differential Evolution (DE), Genetic Algorithm (GA), and Lion Optimization. Ten workload configurations ranging from **5,000 to 50,000 tasks** are evaluated under common experimental conditions. The performance is assessed using latency, energy consumption, throughput, cost, load, SLA, fitness, and runtime. The results show that optimization-based approaches generally improve task-offloading performance compared with the Baseline. GA provides the strongest average performance for latency, response, throughput, load, SLA, and fitness, while DE provides the lowest average energy consumption and cost. The proposed Hybrid GA–Lion approach provides a competitive multi-objective solution, achieving substantial improvements over the Baseline while maintaining balanced performance across the evaluated metrics. Friedman and Wilcoxon statistical tests further demonstrate that the observed differences among the evaluated approaches are statistically significant at the 5% significance level.

Keywords— Mobile Edge Computing, task offloading, Hybrid Genetic Algorithm, Lion Optimization, Hybrid GA–Lion, metaheuristic optimization, performance enhancement

List of Figures

- Fig. 1. Cloud–edge task offloading workflow
- Fig. 2. Response time comparison
- Fig. 3. Energy consumption comparison
- Fig. 4. Throughput comparison

List of Tables

- Table 1. Workload configurations
- Table 2. Optimization strategies
- Table 3. Task properties
- Table 4. VM/server properties
- Table 5. Experimental software and evaluation settings
- Table 6. Mean performance across workloads
- Table 7. Friedman statistical test
- Table 8. Wilcoxon signed-rank comparison

Final Report of Internship Programme

1. Introduction

Previous research has established that edge computing places computation and storage resources closer to users and data sources, making it suitable for latency-sensitive and computation-intensive applications [1], [2]. Research on mobile edge computing also identifies computation offloading as an important mechanism for managing limited edge resources and communication constraints [2], [3].

Existing studies formulate task offloading and resource allocation as optimization problems because workload demands, resource capacities, and communication conditions can vary [3], [4]. Previous work has investigated different optimization and learning-based strategies for improving objectives such as latency, energy consumption, and resource utilization [5]–[10]. These findings are taken from the cited literature; they are not presented as findings of the present project.

Objectives

- Develop a simulation framework for cloud/edge task offloading.
- Evaluate workloads from 5,000 to 50,000 tasks under common conditions.
- Compare Baseline, PSO, DE, GA, Lion, and Hybrid GA–Lion.
- Measure latency, energy, throughput, cost, load, SLA, fitness, and runtime.
- Use statistical tests to determine whether observed differences are systematic.

Final Report of Internship Programme

2 Literature Review

Edge computing places computation closer to data sources and can reduce dependence on distant cloud resources [1], [2]. Task offloading and resource allocation have therefore been formulated as optimization problems under heterogeneous computing and communication constraints [3], [4]. Learning-based approaches have also been studied for dynamic decision spaces, while evolutionary and swarm methods provide population-based search mechanisms [5]–[11].

Scheduling studies show that response time, energy, throughput, fairness, and resource use can conflict [6], [7]. Hybrid metaheuristics are useful when a single search mechanism does not provide sufficient balance. The present project follows this multi-objective perspective and compares several optimization strategies under the same simulated workload conditions.

2.1 Comparative Study of Twenty Relevant Studies

The literature review below summarizes twenty representative studies related to edge computing, mobile edge computing (MEC), computation offloading, resource allocation, task scheduling, and optimization. The statements describe findings reported by the cited researchers and are not presented as results of the present project.

Zhou and Jadoon [17].

The authors investigated partial computation offloading for multiple users using game theory. The study modeled offloading overhead and derived a low-complexity mechanism based on Nash-equilibrium analysis.

Li and Zhu [18].

The authors proposed a Genetic Algorithm for joint optimization of task-offloading proportion, channel bandwidth, and MEC computing resources. This work is relevant to GA-based optimization used for comparison in this project.

Wei et al. [19].

The study investigated computation-offloading decisions under stochastic wireless and computing conditions and used deep Q-learning to learn offloading policies.

Huang et al. [20].

The authors considered security- and cost-aware computation offloading and formulated the decision problem using a Markov decision process and deep reinforcement learning.

Meng et al. [21].

The study addressed task migration in MEC using deep reinforcement learning and considered user mobility and changing suitability of edge servers.

You and Tang [22].

The authors applied Particle Swarm Optimization to task offloading for industrial IoT edge computing and investigated offloading delay under edge-computing constraints.

Huang et al. [23].

The authors proposed a Multi-Objective Whale Optimization Algorithm for MEC computation offloading and considered time and energy simultaneously.

Shakarami et al. [24].

This survey reviewed machine-learning approaches to computation offloading and emphasized heterogeneous hardware, dynamic conditions, and stochastic behavior.

Lin et al. [25].

This survey reviewed computation-offloading models for edge computing, including computation, communication, channel, and energy models and several optimization approaches.

Saeik et al. [26].

The survey compared mathematical optimization, artificial intelligence, and control-theory solutions for task offloading and resource management.

GPU-based PSO study [27].

A GPU-based PSO framework was studied for high-performance vehicular edge computing, considering workload, resource utilization, vehicle movement, and offloading latency.

Final Report of Internship Programme

Jiao et al. [28].

The authors investigated multi-agent deep reinforcement learning for MEC offloading and resource allocation in dynamic multi-device environments.

Collaborative MAPPO study [29].

A collaborative multi-agent reinforcement-learning approach was investigated for joint task offloading and resource allocation in heterogeneous 5G edge networks.

Enhanced Whale Optimization study [30].

A 2024 study proposed an Enhanced Whale Optimization Algorithm for dependent-task offloading in multi-edge cloud computing, jointly considering latency, energy, and cost.

Aminu et al. [31].

The systematic review examined 103 metaheuristic-based task-scheduling studies in edge computing and identified adaptability, network awareness, scalability, and hybrid methods as continuing challenges.

Green edge–cloud offloading study [32].

A recent study used multi-agent reinforcement learning with graph representations for green edge–cloud computation offloading, targeting energy and response delay.

DQN joint-offloading study [33].

A deep-reinforcement-learning method was studied for joint task offloading and bandwidth allocation in multi-user MEC, targeting energy, computation, and delay costs.

Mobile-cloudlet response-time study [34].

A game-theoretic load-balancing approach was investigated for mobile cloudlets with the objective of reducing response time while considering device availability and battery life.

Vehicular edge DRL study [35].

The study developed an intelligent offloading system for vehicular edge computing using deep reinforcement learning and considered communication and computation states.

Hybrid PSO–GWO study [36].

A hybrid PSO–GWO strategy was investigated for resource-intensive mobile applications, jointly considering energy consumption, capacity, delay, and resource allocation.

Literature synthesis. Across these studies, latency, energy consumption, resource utilization, cost, throughput, and service quality repeatedly appear as important objectives. The research direction has progressed from mathematical and game-theoretic formulations toward machine learning, swarm intelligence, genetic algorithms, and hybrid optimization. Recent studies continue to emphasize dynamic environments, scalability, and multi-objective optimization as important challenges [24]–[36].

2.2 Inference Drawn from Literature Review

The reviewed literature provides the basis for the present project. It supports treating task offloading as a multi-objective optimization problem and using established population-based algorithms as comparative methods. Accordingly, this project implements and compares Baseline, PSO, DE, GA, Lion, and Hybrid GA–Lion under common simulated workload configurations. The report does not claim that the basic MEC problem or the individual optimization algorithms were newly developed here; the project-specific contribution is the implemented hybrid approach and its comparative evaluation.

3 Problem Formulation and Proposed Solution

3.1 Overview

The problem is formulated as selecting task execution locations and resource assignments so that overall performance is improved under heterogeneous edge and cloud capacities. Each task has computational and resource requirements, while each server has finite CPU, memory, bandwidth, processing speed, and task-capacity limits.

3.2 System Model

The simulation uses an edge–cloud environment containing an Edge Server and a Cloud Server. The Edge Server represents a nearby resource with lower capacity, while the Cloud Server provides higher processing, memory, bandwidth, and task capacity.

Final Report of Internship Programme

3.3 Task Properties and VM/Server Properties

The simulation models heterogeneous tasks so that each workload contains tasks with different computational and communication requirements. Each generated task contains the properties listed in Table 3. These values are simulation parameters used to represent heterogeneous workloads; they are not measurements collected from a physical edge–cloud testbed.

Task property	Setting	Purpose
Task size	1,000–10,000	Represents computational workload size.
CPU demand	10–100	Represents required CPU capacity.
Memory demand	256–4,096 MB	Represents required memory.
Bandwidth requirement	5–100 Mbps	Represents communication requirement.
Priority	1–5	Represents relative task importance.
Deadline	50–500	Represents completion-time constraint.
Arrival information	Randomly generated	Represents task-arrival variation.
Energy-related value	Randomly generated	Used in the simulation's energy evaluation.

Table 3: Task properties used in the simulation.

The computing resources are represented by two simulated servers. The Edge Server represents a nearby resource with lower capacity, while the Cloud Server represents a higher-capacity remote resource. The server parameters are fixed for all algorithms so that performance differences are attributable to the offloading strategies rather than changing hardware conditions.

VM/Server property	Edge Server	Cloud Server	Description
CPU capacity	100	500	Available CPU capacity.
Memory	4,096 MB	32,768 MB	Available memory.
Bandwidth	1,000 Mbps	10,000 Mbps	Available network bandwidth.
Processing speed	120	250	Relative processing speed used by simulator.
Maximum tasks	150	1,000	Maximum task capacity.

Table 4: VM/server properties used in the simulation.

The task generator produces heterogeneous tasks with size, CPU demand, memory demand, bandwidth requirement, priority, deadline, arrival information, and energy-related values. Workload size is varied from 5,000 to 50,000 tasks, and the same configurations are supplied to every algorithm so that the comparison remains paired.

3.4 Optimization Algorithms

PSO updates candidate solutions using particle movement influenced by individual and collective search experience [12]. DE generates trial solutions using differential mutation, crossover, and selection [13]. GA evolves candidates through selection, crossover, and mutation [14]. Lion Optimization performs population-based lion-inspired search [15]. The Baseline provides the non-metaheuristic reference used for comparison.

3.5 Hybrid GA–Lion Approach

The Hybrid GA–Lion method combines GA-based evolutionary variation with Lion-based population refinement. The objective is to improve search quality by combining complementary optimization behavior. The final evaluation measures whether the additional search effort provides useful multi-objective benefits.

3.6 Evaluation Objective

The evaluation reports each metric separately because the metrics have different units and optimization directions. Lower values are preferred for latency, energy, cost, load, SLA, fitness, and runtime, while higher throughput is preferred.

Final Report of Internship Programme

4 Results and Discussion

The experimental results compare six task offloading approaches over ten workload configurations from 5,000 to 50,000 tasks. The analysis covers response time, energy consumption, throughput, cost, load, SLA, fitness, and runtime.

4.1 Experimental Setup

The experiments were implemented as a Python-based simulation. All six methods use the same generated workload configurations, task properties, server capacities, and evaluation procedure. The following settings describe the experimental environment and software used for the reported results.

Experimental property	Setting
Programming language	Python 3.12
Data processing	Pandas, NumPy
Visualization	Matplotlib
Statistical analysis	SciPy
Workload range	5,000–50,000 tasks
Workload step	5,000 tasks
Number of workload configurations	10
Compared methods	Baseline, PSO, DE, GA, Lion, Hybrid GA–Lion
Computing model	1 Edge Server + 1 Cloud Server
Evaluation measures	Latency, energy, throughput, cost, load, SLA, fitness, runtime
Significance level	$\alpha = 0.05$

Table 5: Experimental software and evaluation settings.

Workload

Task configurations

Low

5,000; 10,000; 15,000

Medium

20,000; 25,000; 30,000; 35,000

High

40,000; 45,000; 50,000

Table 1: Workload configurations used for evaluation.

For every workload, the same six methods are evaluated using the recorded performance metrics. This paired structure allows statistical comparison across the same workload configurations.

4.2 Performance Analysis

Response time increases as workload size grows for all approaches. The Baseline rises from 2,158.44 at 5,000 tasks to 48,930.64 at 50,000 tasks. At 50,000 tasks, Hybrid GA–Lion records 46,338.26. Across the ten configurations, GA has the lowest mean response time at 21,107.22, while Hybrid records 21,137.46.

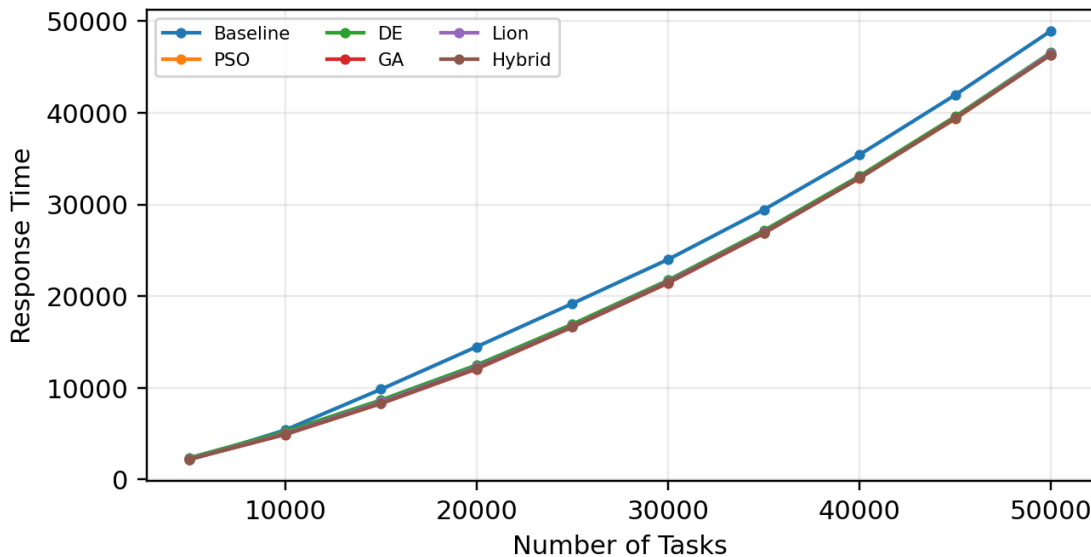


Fig. 2. Response time comparison across workload sizes.

Final Report of Internship Programme

Energy consumption also increases with workload. The Baseline rises from 168.07 to 1,317.04, while DE has the lowest mean energy at 608.25. Hybrid has a mean of 631.47 and records 1,133.29 at 50,000 tasks, below the Baseline. Throughput increases with workload, and GA has the highest mean throughput at 19,816.12; Hybrid records 19,777.39.

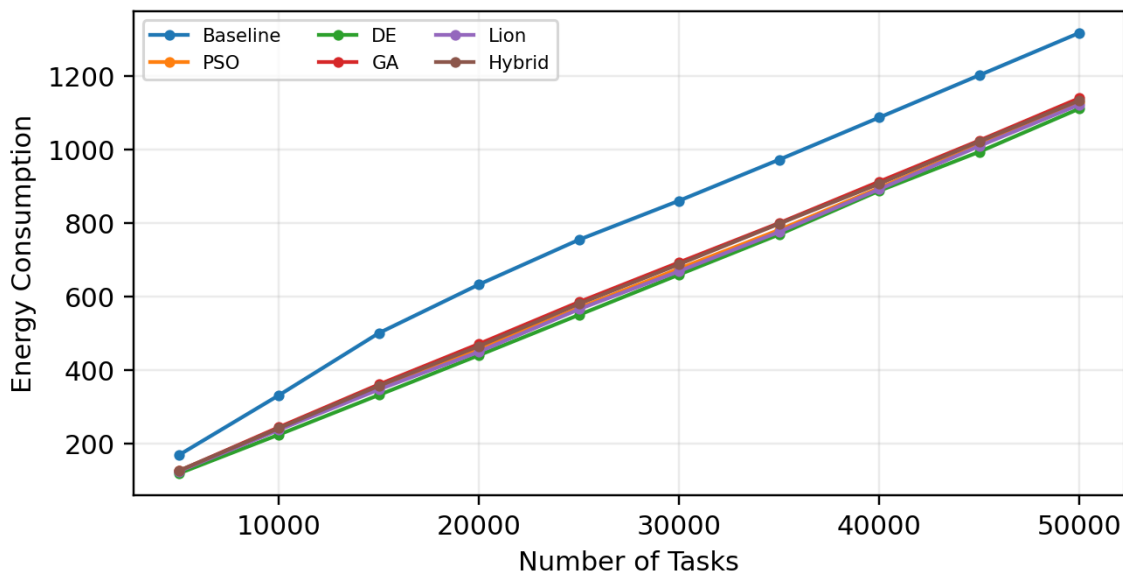


Fig. 3. Energy consumption comparison across workload sizes.

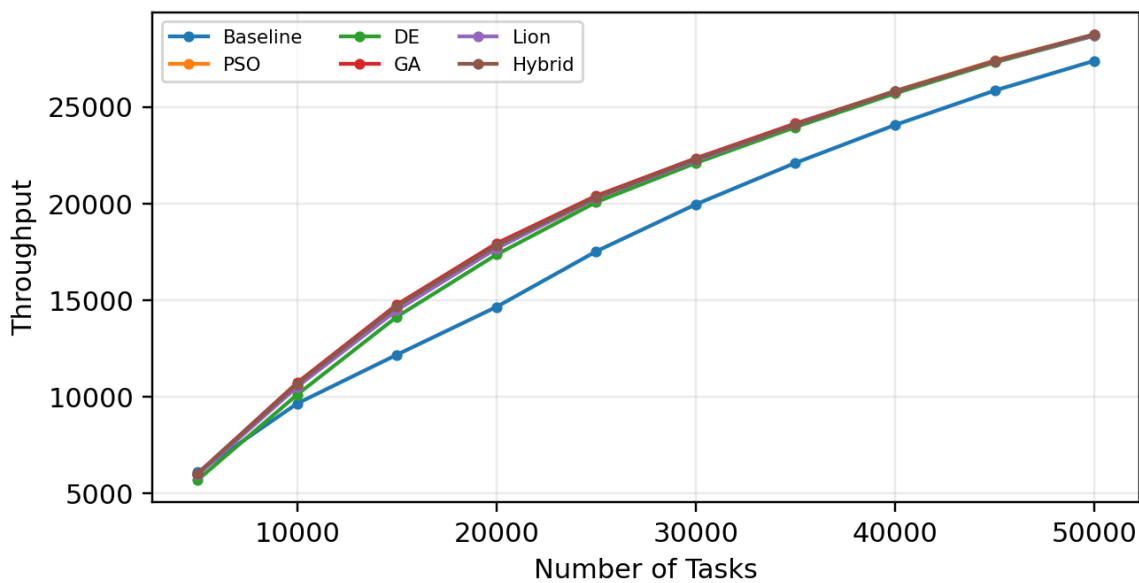


Fig. 4. Throughput comparison across workload sizes.

Final Report of Internship Programme

4.2 Performance Analysis (continued)

The remaining metrics demonstrate that the algorithms have different strengths. GA records the lowest mean load, SLA, and fitness. DE records the lowest mean cost. Hybrid remains competitive across the task-performance metrics but has greater runtime because of its additional optimization process.

Metric	Baseline	PSO	DE	GA	Lion	Hybrid	Best
Latency	23080.65	21160.97	21385.41	21107.22	21208.44	21137.46	GA
Energy	782.41	623.10	608.25	635.27	618.86	631.47	DE
Throughput	17938.88	19739.08	19502.97	19816.12	19701.21	19777.39	GA
Cost	2727.04	1933.60	1842.08	2002.10	1907.84	1980.01	DE
Load	34.01	2.45	4.16	0.94	2.89	1.35	GA
SLA	73.08	52.27	60.24	51.43	53.25	51.58	GA
Fitness	0.1501	0.1023	0.1117	0.1004	0.1034	0.1010	GA
Runtime	0.0375	1.2247	2.4936	6.4233	1.7402	4.1573	Baseline

Table 6: Mean performance across the ten workload configurations.

Relative to the Baseline, the Hybrid method shows average improvements of approximately 8.42% in latency, 20.83% in energy, 10.25% in throughput, 27.41% in cost, 96.03% in load, 29.42% in SLA, and 32.72% in fitness. Runtime is the exception because Hybrid performs additional optimization.

4.3 Statistical Analysis

Because the same ten workload configurations are evaluated for every algorithm, the Friedman test is used as a non-parametric test for comparing multiple methods across matched datasets. The two-sided Wilcoxon signed-rank test is used for the paired comparison between Hybrid GA–Lion and the Baseline. This selection follows the statistical comparison procedure recommended by Demšar for multiple methods over multiple datasets and for paired comparison of two methods [16]. A significance level of $\alpha = 0.05$ is used.

Metric	Friedman Statistic	p-value	Significant ($\alpha=0.05$)
Latency	35.8857	1.00114e-06	Yes
Energy	48.4571	2.86497e-09	Yes
Throughput	34.4000	1.98193e-06	Yes
Response	35.8857	1.00114e-06	Yes
Cost	49.4857	1.76565e-09	Yes
Load	50.0000	1.38589e-09	Yes
SLA	46.4650	7.30289e-09	Yes
Fitness	50.0000	1.38589e-09	Yes
Runtime	50.0000	1.38589e-09	Yes

Table 7: Friedman test for differences among the six algorithms.

Final Report of Internship Programme

All Friedman-test p-values are below 0.05. Therefore, the null hypothesis of identical performance distributions across the six algorithms is rejected for every evaluated metric at the 5% significance level. The Wilcoxon results also give p-values below 0.05 for the Hybrid GA–Lion versus Baseline comparison. Following the statistical methodology in [16], these results provide evidence that the observed differences are systematic across the tested workload configurations.

Metric	Wilcoxon Statistic	p-value	Significant ($\alpha=0.05$)
Latency	1.00	0.00390625	Yes
Energy	0.00	0.00195312	Yes
Throughput	1.00	0.00390625	Yes
Response	1.00	0.00390625	Yes
Cost	0.00	0.00195312	Yes
Load	0.00	0.00195312	Yes
SLA	0.00	0.00195312	Yes
Fitness	0.00	0.00195312	Yes
Runtime	0.00	0.00195312	Yes

Table 8: Wilcoxon signed-rank comparison of Hybrid GA–Lion and Baseline.

The Wilcoxon test also gives p-values below 0.05 for every metric. Therefore, Hybrid and Baseline differ significantly across the tested workload configurations. The direction is metric-dependent: Hybrid improves the main task-performance measures but requires higher runtime.

4.4 Overall Discussion

The results show that no single optimizer is best for every metric. GA is strongest for average latency, response, throughput, load, SLA, and fitness, while DE is strongest for energy and cost. Hybrid GA–Lion provides a balanced multi-objective compromise and substantial reductions relative to the Baseline. The statistical tests support the consistency of these differences, while the runtime results highlight the computational trade-off introduced by optimization.

5 Conclusion and Future Work

5.1 Conclusion

This project developed and evaluated a simulation-based cloud task offloading framework using Baseline, PSO, DE, GA, Lion, and Hybrid GA–Lion. Ten workload configurations from 5,000 to 50,000 tasks were evaluated under common conditions. The results demonstrate that optimization-based task allocation can improve several performance measures relative to the Baseline. The proposed Hybrid Genetic Algorithm and Lion Optimization based task offloading approach demonstrates that combining two complementary metaheuristic strategies can provide a balanced solution for performance enhancement in MEC environments.

The comparison also shows that algorithm performance is objective-dependent. GA provides the best average latency, response, throughput, load, SLA, and fitness values, while DE provides the best average energy and cost. Hybrid GA–Lion offers a competitive multi-objective compromise, although its runtime is higher. The Friedman and Wilcoxon tests provide statistical evidence of significant differences at the 5% level.

5.2 Future Work

- Repeat every workload with multiple independent random seeds and report confidence intervals and effect sizes.
- Extend the workload range and introduce held-out workload regimes for generalization testing.
- Add network delay, queue delay, and dynamic server utilization to the simulation.
- Investigate adaptive optimizer selection based on workload and system state.
- Evaluate the framework on a larger heterogeneous edge–cloud testbed.
- Develop an interactive dashboard for monitoring task allocation and performance.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/JIOT.2016.2579198.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A Survey on Mobile Edge Computing: The Communication Perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017, doi: 10.1109/COMST.2017.2745201.
- [3] P. Mach and Z. Becvar, "Mobile Edge Computing: A Survey on Architecture and Computation Offloading," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017, doi: 10.1109/COMST.2017.2682318.
- [4] S. Sardellitti, G. Scutari, and S. Barbarossa, "Joint Optimization of Radio and Computational Resources for Multicell Mobile-Edge Computing," *IEEE Transactions on Signal and Information Processing over Networks*, vol. 1, no. 2, pp. 89–103, 2015, doi: 10.1109/TSIPN.2015.2448520.
- [5] C. You, K. Huang, H. Chae, and B.-H. Kim, "Energy-Efficient Resource Allocation for Mobile-Edge Computation Offloading," *IEEE Transactions on Wireless Communications*, vol. 16, no. 3, pp. 1397–1411, 2017, doi: 10.1109/TWC.2016.2633522.
- [6] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic Computation Offloading for Mobile-Edge Computing with Energy Harvesting Devices," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3590–3605, 2016, doi: 10.1109/JSAC.2016.2611964.
- [7] M. A. Khan, "A Survey of Computation Offloading Strategies for Performance Improvement of Applications Running on Mobile Devices," *Journal of Network and Computer Applications*, vol. 56, pp. 28–40, 2015, doi: 10.1016/j.jnca.2015.05.018.
- [8] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile Edge Computing: A Survey," *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 450–465, 2018.
- [9] K. Peng, V. C. M. Leung, X. Xu, L. Zheng, J. Wang, and Q. Huang, "A Survey on Mobile Edge Computing: Focusing on Service Adoption and Provision," *Wireless Communications and Mobile Computing*, vol. 2018, Art. no. 8267838, 2018, doi: 10.1155/2018/8267838.
- [10] K. Akherfi, M. Gerndt, and H. Harroud, "Mobile Cloud Computing for Computation Offloading: Issues and Challenges," *Applied Computing and Informatics*, vol. 14, no. 1, pp. 1–16, 2018, doi: 10.1016/j.aci.2016.11.002.
- [11] C. Feng, P. Han, X. Zhang, B. Yang, Y. Liu, and L. Guo, "Computation Offloading in Mobile Edge Computing Networks: A Survey," *Journal of Network and Computer Applications*, vol. 202, Art. no. 103366, 2022, doi: 10.1016/j.jnca.2022.103366.
- [12] J. Kennedy and R. Eberhart, "Particle Swarm Optimization," in *Proceedings of the IEEE International Conference on Neural Networks (ICNN'95)*, Perth, Australia, 1995, pp. 1942–1948, doi: 10.1109/ICNN.1995.488968.
- [13] R. Storn and K. Price, "Differential Evolution—A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997, doi: 10.1023/A:1008202821328.
- [14] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.
- [15] M. Yazdani and F. Jolai, "Lion Optimization Algorithm (LOA): A Nature-Inspired Metaheuristic Algorithm," *Journal of Computational Design and Engineering*, vol. 3, no. 1, pp. 24–36, 2016, doi: 10.1016/j.jcde.2015.06.003.
- [16] J. Demšar, "Statistical Comparisons of Classifiers over Multiple Data Sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [17] S. Zhou and W. Jadoon, "The Partial Computation Offloading Strategy Based on Game Theory for Multi-User in Mobile Edge Computing Environment," *Computer Networks*, vol. 159, pp. 112–127, 2019.
- [18] Z. Li and Q. Zhu, "Genetic Algorithm-Based Optimization of Offloading and Resource Allocation in Mobile-Edge Computing," *Information*, vol. 11, no. 2, 83, 2020.
- [19] Y. Wei, Z. Wang, D. Guo, and F. R. Yu, "Deep Q-Learning Based Computation Offloading Strategy for Mobile Edge Computing," *Computers, Materials & Continua*, vol. 59, no. 1, pp. 89–104, 2019.
- [20] B. Huang, Y. Li, Z. Li, L. Pan, and Y. Xu, "Security and Cost-Aware Computation Offloading via Deep Reinforcement Learning in Mobile Edge Computing," *Wireless Communications and Mobile Computing*, 2019.
- [21] F. Meng et al., "Task Migration for Mobile Edge Computing Using Deep Reinforcement Learning," *Future Generation Computer Systems*, 2019.

Final Report of Internship Programme

- [22] Q. You and B. Tang, "Efficient Task Offloading Using Particle Swarm Optimization Algorithm in Edge Computing for Industrial Internet of Things," *Journal of Cloud Computing*, vol. 10, article 41, 2021.
- [23] M. Huang, Q. Zhai, Y. Chen, S. Feng, and F. Shu, "Multi-Objective Whale Optimization Algorithm for Computation Offloading Optimization in Mobile Edge Computing," *Sensors*, vol. 21, no. 8, 2628, 2021.
- [24] A. Shakarami, M. Ghobaei-Arani, and A. Shahidinejad, "A Survey on the Computation Offloading Approaches in Mobile Edge Computing: A Machine Learning-Based Perspective," *Computer Networks*, vol. 182, 107496, 2020.
- [25] H. Lin, S. Zeadally, Z. Chen, H. Labiod, and L. Wang, "A Survey on Computation Offloading Modeling for Edge Computing," *Journal of Network and Computer Applications*, vol. 169, 102781, 2020.
- [26] F. Saeik et al., "Task Offloading in Edge and Cloud Computing: A Survey on Mathematical, Artificial Intelligence and Control Theory Solutions," *Computer Networks*, vol. 195, 108177, 2021.
- [27] "Task Offloading Using GPU-Based Particle Swarm Optimization for High-Performance Vehicular Edge Computing," *Journal of King Saud University—Computer and Information Sciences*, vol. 34, 2022.
- [28] T. Jiao, X. Feng, C. Guo, D. Wang, and J. Song, "Multi-Agent Deep Reinforcement Learning for Efficient Computation Offloading in Mobile Edge Computing," *Computers, Materials & Continua*, vol. 76, no. 3, pp. 3585–3603, 2023.
- [29] "A Collaborative Optimization Strategy for Computing Offloading and Resource Allocation Based on Multi-Agent Deep Reinforcement Learning," *Computers & Electrical Engineering*, 2022.
- [30] "Enhanced Whale Optimization Algorithm for Dependent Tasks Offloading Problem in Multi-Edge Cloud Computing," *Alexandria Engineering Journal*, vol. 97, pp. 302–318, 2024.
- [31] J. Aminu, R. Latip, Z. M. Hanafi, S. Kamarudin, and D. Gabi, "Systematic Review of Metaheuristic-Based Task Scheduling Strategies in Edge Computing Environments," *Discover Computing*, vol. 28, article 307, 2025.
- [32] "Multi-Agent Reinforcement Learning with Graph Representation for Green Edge–Cloud Computation Offloading," *Computer Communications*, 2025.
- [33] L. Huang, X. Feng, C. Zhang, L. Qian, and Y. Wu, "Deep Reinforcement Learning-Based Joint Task Offloading and Bandwidth Allocation for Multi-User Mobile Edge Computing," *Digital Communications and Networks*, vol. 5, no. 1, pp. 10–17, 2019.
- [34] M. Tiwary, D. Puthal, K. S. Sahoo, B. Sahoo, and L. T. Yang, "Response Time Optimization for Cloudlets in Mobile Edge Computing," *Journal of Parallel and Distributed Computing*, vol. 119, pp. 81–91, 2018.
- [35] Z. Ning, P. Dong, X. Wang, J. J. P. C. Rodrigues, and F. Xia, "Deep Reinforcement Learning for Vehicular Edge Computing: An Intelligent Offloading System," *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 6, article 60, 2019.
- [36] "Energy-Efficient Task Offloading Strategy in Mobile Edge Computing for Resource-Intensive Mobile Applications," *Digital Communications and Networks*, 2022.